

Name:-Krish Satpal

Roll no:-28

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

1. Importing Required Libraries

- **datetime** → to timestamp each block
- **hashlib** → to generate SHA-256 hashes
- **json** → to encode blocks as JSON for hashing
- **Flask** → to run the blockchain API server

2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
 - store the chain
 - implement Proof of Work
 - validate chain integrity
-

3. Flask Web API

Provides endpoints:

- `/mine_block` → mine a new block
 - `/get_chain` → fetch full blockchain
 - `/is_valid` → verify blockchain
-

CODE

Step 1

```
import datetime
import hashlib
import json

class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = [] # Initialize transactions list
        self.create_block(proof = 1, previous_hash = '0')

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                  'timestamp': str(datetime.datetime.now()),
                  'proof': proof,
```

Name:-Krish Satpal

Roll no:-28

```
        'previous_hash': previous_hash,
        'transactions': self.transactions} # Include transactions
in the block
        self.transactions = [] # Clear transactions after adding to a
block
        self.chain.append(block)
        return block

def get_previous_block(self):
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    new_proof = 1
    check_proof = False
    while check_proof is False:
        # Adjusting the cryptographic puzzle to '000' leading zeros
        hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
        if hash_operation[:3] == '000': # Changed from '0000' to '000'
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys = True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False, f"Blockchain invalid: Block {block['index']}
has an incorrect previous hash."
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdigest()
```

Name:-Krish Satpal

Roll no:-28

```
        if hash_operation[:3] != '000': # Changed from '0000' to '000'
            return False, f"Blockchain invalid: Block {block['index']}
has an invalid proof of work."
        previous_block = block
        block_index += 1
        return True, "All good. The Blockchain is valid."

# New method to create a transaction
def add_transaction(self, sender, receiver, amount):
    self.transactions.append({
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    })
    # Return the index of the block where this transaction will be
added next
    return self.get_previous_block()['index'] + 1
```

Step 2

Creating a Blockchain instance

```
blockchain = Blockchain()
```

Mining a new block

```
def mine_block():
    if not blockchain.transactions:
        return {'message': 'No transactions to mine. Add some transactions
first.'}

    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash_value = blockchain.hash(previous_block) # Store previous
block's hash
    block = blockchain.create_block(proof, previous_hash_value) # Pass the
hash of the previous block
    current_block_hash = blockchain.hash(block) # Calculate the hash of
the newly created block

    response = {'message': 'Congratulations, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
```

Name:-Krish Satpal

Roll no:-28

```
        'proof': block['proof'],
        'previous_hash': previous_hash_value, # This is the hash
of the *actual* previous block
        'current_block_hash': current_block_hash, # New: hash of
the current block
        'transactions': block['transactions']}
    return response

# Getting the full Blockchain
def get_chain():
    chain_with_hashes = []
    for block in blockchain.chain:
        block_copy = block.copy() # Create a copy to add
current_block_hash without modifying original
        block_copy['current_block_hash'] = blockchain.hash(block)
        chain_with_hashes.append(block_copy)
    response = {'chain': chain_with_hashes,
                'length': len(blockchain.chain)}
    return response

# Checking if the Blockchain is valid
def is_valid():
    is_valid_chain, message = blockchain.is_chain_valid(blockchain.chain)
# Get message from class method
    response = {'message': message} # Use the detailed message directly
    return response
```

Step 3

```
print("Functions Menu:")
print("=====")
print("1. Mine a block")
print("2. Display the chain")
print("3. Check the validity of the chain")
print("4. Add a transaction")
choice = int(input("Enter your choice :"))

if choice == 1:
    print(mine_block())
elif choice == 2:
    print(get_chain())
elif choice == 3:
```

Name:-Krish Satpal

Roll no:-28

```
print(is_valid())
elif choice == 4:
    sender = input("Enter sender: ")
    receiver = input("Enter receiver: ")
    amount = float(input("Enter amount: "))
    index = blockchain.add_transaction(sender, receiver, amount)
    print(f'Transaction added and will be added to Block {index}')
else :
    print("Invalid choice")
```

Output:

Functions Menu:

=====

1. Mine a block
2. Display the chain
3. Check the validity of the chain
4. Add a transaction

Enter your choice :

1. Add a transaction

Functions Menu:

=====

1. Mine a block
2. Display the chain
3. Check the validity of the chain
4. Add a transaction

Enter your choice :4

Enter sender: krish

Enter receiver: bhavik

Enter amount: 50000

Transaction added and will be added to Block 3

2.Mine a block

Functions Menu:

=====

1. Mine a block
2. Display the chain
3. Check the validity of the chain
4. Add a transaction

Enter your choice :1

```
{'message': 'Congratulations, you just mined a block!', 'index': 3,
'timestamp': '2026-08-24 06:39:16.535913', 'proof': 11028,
'previous_hash':
'd8a1516ded822c9ab8e6611fcebada89ba0cfa426bb0afc8ea8a299bf40d57d65',
```

Name:-Krish Satpal

Roll no:-28

```
'current_block_hash':  
'1ec2da50e07b020a4e8bca52709d0fd92f9e81aafe90239d91e8f41d706db37d',  
'transactions': [{ 'sender': 'krish ', 'receiver': 'bhavik', 'amount':  
50000.0}]]}
```

3.Display the chain

Functions Menu:

=====

1. Mine a block
2. Display the chain
3. Check the validity of the chain
4. Add a transaction

Enter your choice :2

```
{ 'chain': [{ 'index': 1, 'timestamp': '2026-08-24 06:28:09.006800',  
'proof': 1, 'previous_hash': '0', 'transactions': [],  
'current_block_hash':  
'20585a464922d22d4d8cc45b2f13a8ab91e0696f283badc0cb5cc65d3db54835'},  
{ 'index': 2, 'timestamp': '2026-08-24 06:29:25.714787', 'proof': 533,  
'previous_hash':  
'20585a464922d22d4d8cc45b2f13a8ab91e0696f283badc0cb5cc65d3db54835',  
'transactions': [{ 'sender': 'krish ', 'receiver': 'bhavik', 'amount':  
50100.0}], 'current_block_hash':  
'd8a1516ded822c9ab8e6611fcebada89ba0cfa426bb0afc8ea8a299bf40d57d65'},  
{ 'index': 3, 'timestamp': '2026-08-24 06:39:16.535913', 'proof': 11028,  
'previous_hash':  
'd8a1516ded822c9ab8e6611fcebada89ba0cfa426bb0afc8ea8a299bf40d57d65',  
'transactions': [{ 'sender': 'krish ', 'receiver': 'bhavik', 'amount':  
50000.0}], 'current_block_hash':  
'1ec2da50e07b020a4e8bca52709d0fd92f9e81aafe90239d91e8f41d706db37d'}]],  
'length': 3}
```

4.check the validity of the chain

Functions Menu:

=====

1. Mine a block
2. Display the chain
3. Check the validity of the chain
4. Add a transaction

Enter your choice :3

```
{ 'message': 'All good. The Blockchain is valid.'}
```

Name:-Krish Satpal

Roll no:-28

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.